

Ejercicio 6: Decoración Óptima de Cuadros mediante Algoritmos Voraces

Este documento presenta el diseño, justificación y análisis de un algoritmo Voraz (*Greedy*) óptimo para colocar el mínimo número de cuadros en un pasillo, asegurando que cada puerta tenga al menos un cuadro a una distancia menor o igual a 0.75 metros.

1. Estrategia Algorítmica (Elección Voraz)

El problema consiste en cubrir un conjunto de puntos (posiciones de las puertas) en una recta utilizando el menor número de intervalos de radio 0.75 metros (es decir, de diámetro 1.5 metros).

Propiedad de la Elección Voraz

Supongamos que las posiciones de las n puertas están ordenadas de izquierda a derecha en el pasillo: $x_1 < x_2 < \dots < x_n$.

Para la primera puerta sin cubrir (inicialmente x_1), debemos colocar un cuadro de tal manera que la cubra. Para maximizar la probabilidad de que este mismo cuadro cubra también a las siguientes puertas situadas a su derecha, debemos colocarlo lo más a la derecha posible de la puerta x_1 .

- Como la distancia máxima permitida entre una puerta y un cuadro es de 0.75 metros, la posición óptima voraz para colocar el primer cuadro es exactamente en:

$$y_1 = x_1 + 0.75$$

- Este cuadro cubrirá automáticamente a todas las puertas x_j que se encuentren en el intervalo continuo:

$$[x_1, x_1 + 1.5]$$

- Una vez colocadas las puertas cubiertas, identificamos la primera puerta x_k que quede fuera de este intervalo ($x_k > x_1 + 1.5$) y repetimos el mismo razonamiento colocándole un cuadro en $x_k + 0.75$.

2. Algoritmo en Pseudocódigo

A continuación se presenta el pseudocódigo del algoritmo utilizando indexación base 1:

```

Funcion ColocarMinimosCuadros(x, n)
Inicio
    // 1. Ordenar las posiciones de las puertas de izquierda a derecha
    x_ordenado <- OrdenarAscendente(x, n) // Coste: O(n log n)

    cuadros_posiciones <- Lista vacía // Almacenará las coordenadas de los cuadro
  
```

```

i <- 1 // Puntero para recorrer las puertas

Mientras i <= n Hacer
  // x_ordenado[i] es la primera puerta que todavía está sin cubrir
  // Paso Voraz: Colocar el cuadro lo más a la derecha posible de esta puer
  pos_cuadro <- x_ordenado[i] + 0.75
  Añadir pos_cuadro a cuadros_posiciones

  // Avanzar el puntero hasta encontrar la primera puerta que quede fuera d
  // (es decir, a una distancia mayor de 0.75 m del cuadro recién colocado)
  Mientras i <= n y x_ordenado[i] <= pos_cuadro + 0.75 Hacer
    i <- i + 1
  FinMientras
FinMientras

Retornar cuadros_posiciones
FinFuncion

```

3. Análisis de Coste

- **Complejidad Temporal:** $O(n \log n)$. El paso limitante es la ordenación inicial de las n posiciones de las puertas, que requiere un coste de $O(n \log n)$ utilizando un algoritmo eficiente como *MergeSort*. El bucle `Mientras` recorre las puertas utilizando un puntero que avanza estrictamente de izquierda a derecha sin retroceder nunca, realizando un escaneo lineal de coste $O(n)$.
- **Complejidad Espacial:** $O(n)$ en el peor caso para almacenar las coordenadas de los cuadros resultantes en la lista de salida, además del espacio temporal requerido para la ordenación.

4. Demostración de Optimalidad (Greedy Stays Ahead)

Demostramos que el algoritmo voraz es óptimo utilizando un argumento inductivo de tipo "El algoritmo voraz siempre va por delante" (*Greedy stays ahead*).

Let us denote:

- $G = (g_1, g_2, \dots, g_k)$ las posiciones de los cuadros colocados por nuestro algoritmo voraz.
- $O = (o_1, o_2, \dots, o_m)$ las posiciones de los cuadros colocados por una solución óptima cualquiera, ordenadas de izquierda a derecha.

Enunciado de la hipótesis inductiva

Para todo cuadro j ($1 \leq j \leq k$), la cobertura alcanzada hacia la derecha por el j -ésimo cuadro del algoritmo voraz es mayor o igual que la alcanzada por el j -ésimo cuadro de la solución óptima:

$$g_j + 0.75 \geq o_j + 0.75 \implies g_j \geq o_j$$

Prueba por Inducción

1. **Caso Base ($j = 1$):** El algoritmo voraz detecta la primera puerta en x_1 y coloca su cuadro en $g_1 = x_1 + 0.75$. Cualquier solución óptima válida O debe cubrir obligatoriamente la puerta x_1 , por lo que su primer cuadro no puede situarse más a la derecha de $x_1 + 0.75$. Por tanto:

$$o_1 \leq x_1 + 0.75 = g_1 \implies g_1 \geq o_1 \quad (\text{Se cumple el caso base})$$

2. **Paso Inductivo:** Supongamos que la hipótesis es cierta para el cuadro $j - 1$, es decir, $g_{j-1} \geq o_{j-1}$. Queremos probar que se cumple para el cuadro j :
 - Como $g_{j-1} \geq o_{j-1}$, la cobertura del cuadro voraz $j - 1$ llega al menos tan lejos a la derecha como la del cuadro óptimo $j - 1$.
 - Sea x_s la primera puerta que queda sin cubrir tras aplicar los primeros $j - 1$ cuadros del algoritmo voraz. Dado que el voraz ha cubierto más o igual terreno a la derecha, la primera puerta que le queda sin cubrir a la solución óptima (vamos a llamarla x_r) tiene que estar obligatoriamente a la izquierda o coincidir con la del voraz:

$$x_r \leq x_s$$

- El algoritmo voraz colocará su j -ésimo cuadro en $g_j = x_s + 0.75$.
- La solución óptima debe cubrir su puerta restante x_r , por lo que su j -ésimo cuadro no puede situarse a la derecha de $x_r + 0.75$. Por tanto:

$$o_j \leq x_r + 0.75 \leq x_s + 0.75 = g_j \implies g_j \geq o_j$$

Se cumple la hipótesis para todo j . Dado que el algoritmo voraz siempre cubre una zona mayor o igual hacia la derecha con la misma cantidad de cuadros, es imposible que la solución óptima necesite menos cuadros para cubrir el pasillo completo. Por tanto, el número de cuadros del algoritmo voraz es mínimo ($k \leq m$), demostrando su **optimalidad global**.